

Satellite Constellation System

Внедрение кэширования через Redis с Spring Cache Abstraction

Семинар 13

Мартirosян Микаэл Дереникович

13 июня 2026 г.

Содержание

1	Описание проекта	2
1.1	Цели	2
2	Архитектура кэширования	2
2.1	Используемые технологии	2
3	Структура проекта	2
4	Настройка окружения	3
4.1	Docker Compose	3
4.2	Зависимости Gradle	3
4.3	Конфигурация приложения	4
5	Реализация кэширования	5
5.1	Включение кэширования	5
5.2	Конфигурация TTL и обработка ошибок	5
5.3	Аннотации в SatelliteService	7
6	Метрики кэша в Actuator	9
7	Запуск и тестирование	9
7.1	Предварительные требования	9
7.2	Запуск инфраструктуры	9
7.3	Сборка и запуск сервиса	9
7.4	Проверка кэширования	10
7.5	Проверка graceful degradation	10
8	Заключение	10

1 Описание проекта

Спутниковая система состоит из нескольких микросервисов, центральным из которых является `space-operation-center`. Этот сервис часто запрашивает данные о спутниках и группировках, что создаёт нагрузку на базу данных. Для ускорения операций чтения и снижения нагрузки внедряется кэширование с использованием **Redis** и абстракции **Spring Cache**.

1.1 Цели

- Кэшировать результаты методов чтения с разными TTL.
- Автоматически инвалидировать кэш при изменении данных.
- Обеспечить отказоустойчивость: при недоступности Redis приложение должно продолжать работу (graceful degradation).
- Предоставить метрики кэширования через Actuator.

2 Архитектура кэширования

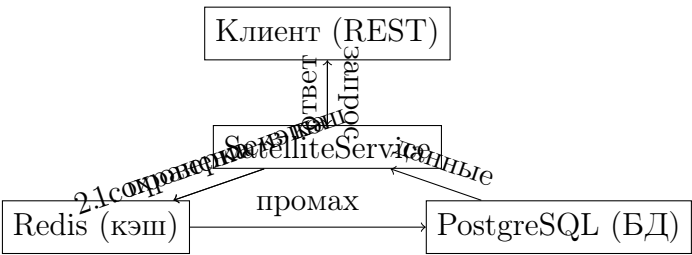


Рис. 1: Поток запроса с кэшем

2.1 Используемые технологии

Технология	Назначение
Spring Cache Abstraction	Декларативное кэширование через аннотации (<code>@Cacheable</code> , <code>@CacheEvict</code>)
Redis	Высокопроизводительное in-memory хранилище для кэша
Spring Boot Actuator	Экспорт метрик кэша (количество чтений/записей/промахов)
Lettuce (клиент Redis)	Асинхронное подключение к Redis, настраивается пул соединений

3 Структура проекта

Ниже представлено дерево каталогов после внедрения кэширования. ✓ – изменённые или новые файлы, остальные – без изменений.

```
satellite-constellation-system/
  docker-compose.yml          ✓ (добавлен Redis)
  README.tex                  ✓ (этот отчёт)
  build.gradle.kts
  settings.gradle.kts
space-operation-center/
  build.gradle.kts            ✓ (зависимости)
```

```

src/main/
  java/com/example/spacecenter/
    SpaceOperationCenterApplication.java ✓ (@EnableCaching)
  config/
    CacheConfig.java ✓ (новый)
  domain/... (без изменений)
  dto/...
  repository/...
  service/
    SatelliteService.java ✓ (аннотации кэша)
    OutboxScheduler.java
  resources/
    application.yml ✓ (настройки Redis)
    schema.sql
telemetry-service/ (без изменений)
mission-scheduler/ (без изменений)

```

4 Настройка окружения

4.1 Docker Compose

Добавляем сервис Redis в `docker-compose.yml`:

Листинг 1: Полный `docker-compose.yml`

```

1 version: '3.8'
2 services:
3   postgres:
4     image: postgres:15
5     environment:
6       POSTGRES_DB: satellites
7       POSTGRES_USER: satuser
8       POSTGRES_PASSWORD: satpass
9     ports:
10      - "5432:5432"
11     volumes:
12      - pgdata:/var/lib/postgresql/data
13
14   redis:
15     image: redis:7-alpine
16     ports:
17      - "6379:6379"
18     volumes:
19      - redisdata:/data
20     command: redis-server --appendonly yes
21
22 volumes:
23   pgdata:
24   redisdata:

```

4.2 Зависимости Gradle

Файл `space-operation-center/build.gradle.kts` дополняется стартерами для кэша и Redis:

Листинг 2: `build.gradle.kts`

```

1 plugins {
2     id("org.springframework.boot") version "3.2.0"
3     id("io.spring.dependency-management") version "1.1.4"
4     java
5 }
6
7 group = "com.example"
8 version = "0.0.1-SNAPSHOT"
9
10 java {
11     sourceCompatibility = JavaVersion.VERSION_17
12 }
13
14 repositories {
15     mavenCentral()
16 }
17
18 dependencies {
19     implementation("org.springframework.boot:spring-boot-starter-web")
20     implementation("org.springframework.boot:spring-boot-starter-data-jpa")
21     implementation("org.springframework.boot:spring-boot-starter-cache")
22     implementation("org.springframework.boot:spring-boot-starter-data-redis")
23     implementation("org.springframework.boot:spring-boot-starter-actuator")
24     runtimeOnly("org.postgresql:postgresql")
25     compileOnly("org.projectlombok:lombok")
26     annotationProcessor("org.projectlombok:lombok")
27     testImplementation("org.springframework.boot:spring-boot-starter-test")
28 }

```

4.3 Конфигурация приложения

application.yml настраивает подключение к Redis, имена кэшей и экспорт метрик:

Листинг 3: application.yml

```

1 spring:
2   datasource:
3     url: jdbc:postgresql://localhost:5432/satellites
4     username: satuser
5     password: satpass
6   jpa:
7     hibernate:
8       ddl-auto: validate
9     show-sql: true
10  sql:
11    init:
12      mode: always
13  cache:
14    type: redis
15    cache-names:
16      - satellite
17      - constellation
18      - satellites
19    redis:
20      time-to-live: 600000 # TTL (
21                             )
22  redis:

```

```

22     host: localhost
23     port: 6379
24     timeout: 2000ms
25     lettuce:
26         pool:
27             max-active: 8
28             max-idle: 8
29             min-idle: 2
30
31 management:
32     endpoints:
33         web:
34             exposure:
35                 include: health,metrics,caches
36     endpoint:
37         metrics:
38             enabled: true
39         caches:
40             enabled: true

```

5 Реализация кэширования

5.1 Включение кэширования

Точка входа приложения аннотирована `@EnableCaching`:

Листинг 4: SpaceOperationCenterApplication.java

```

1 package com.example.spacecenter;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5 import org.springframework.cache.annotation.EnableCaching;
6
7 @SpringBootApplication
8 @EnableCaching
9 public class SpaceOperationCenterApplication {
10     public static void main(String[] args) {
11         SpringApplication.run(SpaceOperationCenterApplication.class, args);
12     }
13 }

```

5.2 Конфигурация TTL и обработка ошибок

Класс `CacheConfig` реализует `CachingConfigurer` и задаёт:

- Разные TTL для кэшей: `satellite` – 10 мин, `constellation` – 15 мин, `satellites` – 5 мин.
- `CacheErrorHandler`, который перехватывает все ошибки Redis и не выбрасывает исключения, обеспечивая плавную деградацию.

Листинг 5: CacheConfig.java

```

1 package com.example.spacecenter.config;
2
3 import org.springframework.cache.CacheManager;

```

```

4 import org.springframework.cache.annotation.CachingConfigurer;
5 import org.springframework.cache.interceptor.CacheErrorHandler;
6 import org.springframework.cache.interceptor.SimpleCacheErrorHandler;
7 import org.springframework.context.annotation.Bean;
8 import org.springframework.context.annotation.Configuration;
9 import org.springframework.data.redis.cache.RedisCacheConfiguration;
10 import org.springframework.data.redis.cache.RedisCacheManager;
11 import org.springframework.data.redis.connection.RedisConnectionFactory;
12 import
    org.springframework.data.redis.serializer.GenericJackson2JsonRedisSerializer;
13 import org.springframework.data.redis.serializer.RedisSerializationContext;
14 import java.time.Duration;
15
16 @Configuration
17 public class CacheConfig implements CachingConfigurer {
18
19     @Bean
20     public CacheManager cacheManager(RedisConnectionFactory
        connectionFactory) {
21         RedisCacheConfiguration defaultConfig =
22             RedisCacheConfiguration.defaultCacheConfig()
23                 .serializeValuesWith(
24                     RedisSerializationContext.SerializationPair.fromSerializer(
25                         new GenericJackson2JsonRedisSerializer()
26                     )
27                 );
28
29         RedisCacheConfiguration satelliteCacheConfig =
30             RedisCacheConfiguration.defaultCacheConfig()
31                 .entryTtl(Duration.ofMinutes(10));
32         RedisCacheConfiguration constellationCacheConfig =
33             RedisCacheConfiguration.defaultCacheConfig()
34                 .entryTtl(Duration.ofMinutes(15));
35         RedisCacheConfiguration satellitesCacheConfig =
36             RedisCacheConfiguration.defaultCacheConfig()
37                 .entryTtl(Duration.ofMinutes(5));
38
39         return RedisCacheManager.builder(connectionFactory)
40             .cacheDefaults(defaultConfig)
41             .withCacheConfiguration("satellite", satelliteCacheConfig)
42             .withCacheConfiguration("constellation",
43                 constellationCacheConfig)
44             .withCacheConfiguration("satellites", satellitesCacheConfig)
45             .build();
46     }
47
48     @Override
49     @Bean
50     public CacheErrorHandler errorHandler() {
51         return new SimpleCacheErrorHandler() {
52             @Override
53             public void handleCacheGetError(RuntimeException exception,
54                 org.springframework.cache.Cache
55                     cache,
56                 Object key) {
57                 System.err.println("Cache GET error for " + cache.getName()
58                     + " key " + key + ": " + exception.getMessage());
59             }
60         };
61     }
62 }

```

```

56     }
57
58     @Override
59     public void handleCachePutError(RuntimeException exception,
60                                     org.springframework.cache.Cache
61                                     cache,
62                                     Object key, Object value) {
63         System.err.println("Cache PUT error for " + cache.getName()
64                             + ": " + exception.getMessage());
65     }
66
67     @Override
68     public void handleCacheEvictError(RuntimeException exception,
69                                      org.springframework.cache.Cache
70                                      cache,
71                                      Object key) {
72         System.err.println("Cache EVICT error for " + cache.getName()
73                             + ": " + exception.getMessage());
74     }
75
76     @Override
77     public void handleCacheClearError(RuntimeException exception,
78                                       org.springframework.cache.Cache
79                                       cache) {
80         System.err.println("Cache CLEAR error for " + cache.getName()
81                             + ": " + exception.getMessage());
82     }
83 }

```

5.3 Аннотации в SatelliteService

Методы сервиса используют `@Cacheable` для чтения, `@CacheEvict` и `@Caching` для инвалидации. Кастомный ключ для поиска по составному имени.

Листинг 6: SatelliteService.java

```

1 package com.example.spacecenter.service;
2
3 import com.example.spacecenter.domain.Constellation;
4 import com.example.spacecenter.domain.Satellite;
5 import com.example.spacecenter.repository.ConstellationRepository;
6 import com.example.spacecenter.repository.SatelliteRepository;
7 import lombok.RequiredArgsConstructor;
8 import org.springframework.cache.annotation.CacheEvict;
9 import org.springframework.cache.annotation.Cacheable;
10 import org.springframework.cache.annotation.Caching;
11 import org.springframework.stereotype.Service;
12 import org.springframework.transaction.annotation.Transactional;
13
14 import java.util.List;
15 import java.util.Optional;
16
17 @Service
18 @RequiredArgsConstructor
19 @Transactional(readOnly = true)

```

```

20 public class SatelliteService {
21
22     private final SatelliteRepository satelliteRepository;
23     private final ConstellationRepository constellationRepository;
24
25     @Cacheable(value = "satellite", key = "#id")
26     public Optional<Satellite> getSatelliteById(Long id) {
27         return satelliteRepository.findById(id);
28     }
29
30     @Cacheable(value = "constellation", key = "#name")
31     public Optional<Constellation> getConstellationByName(String name) {
32         return constellationRepository.findByName(name);
33     }
34
35     @Cacheable(value = "satellites", key = "'all'")
36     public List<Satellite> getAllSatellites() {
37         return satelliteRepository.findAll();
38     }
39
40     @Cacheable(value = "satellite",
41               key = "#constellationName + '::' + #satelliteName")
42     public Optional<Satellite> findByConstellationAndName(
43         String constellationName, String satelliteName) {
44         return satelliteRepository
45             .findByConstellationNameAndName(constellationName,
46                 satelliteName);
47     }
48
49     @Transactional
50     @CacheEvict(value = "satellites", allEntries = true)
51     public Satellite createSatellite(Satellite satellite) {
52         return satelliteRepository.save(satellite);
53     }
54
55     @Transactional
56     @CacheEvict(value = "satellite", key = "#satellite.id")
57     public Satellite updateSatellite(Satellite satellite) {
58         return satelliteRepository.save(satellite);
59     }
60
61     @Transactional
62     @Caching(evict = {
63         @CacheEvict(value = "satellite", key = "#id"),
64         @CacheEvict(value = "satellites", allEntries = true)
65     })
66     public void deleteSatellite(Long id) {
67         satelliteRepository.deleteById(id);
68     }
69
70     @Transactional
71     @Caching(evict = {
72         @CacheEvict(value = "constellation", key = "#name"),
73         @CacheEvict(value = "satellites", allEntries = true)
74     })
75     public void updateConstellationComposition(String name, List<Long>
76         satelliteIds) {

```



```

75     Constellation constellation =
        constellationRepository.findByName(name)
76         .orElseThrow(() -> new
            IllegalArgumentException("Constellation not found"));
77     List<Satellite> satellites =
        satelliteRepository.findAllById(satelliteIds);
78     constellation.setSatellites(satellites);
79     constellationRepository.save(constellation);
80 }
81 }

```

6 Метрики кэша в Actuator

После запуска приложения доступны следующие эндпоинты Actuator:

- `/actuator/metrics/cache.gets` – количество успешных чтений из кэша
- `/actuator/metrics/cache.puts` – количество записей в кэш
- `/actuator/metrics/cache.evictions` – количество удалений (инвалидаций)
- `/actuator/caches` – подробная информация по каждому кэшу (имя, TTL, статистика)

Для их работы в `application.yml` включён экспорт `caches` и `metrics`.

7 Запуск и тестирование

7.1 Предварительные требования

- Docker и Docker Compose
- Java 17
- Gradle (или использование wrapper)

7.2 Запуск инфраструктуры

Выполнить в корне проекта:

Листинг 7: Запуск Docker-контейнеров

```

1 docker-compose up -d

```

Поднимутся PostgreSQL и Redis.

7.3 Сборка и запуск сервиса

Листинг 8: Запуск space-operation-center

```

1 cd space-operation-center
2 ./gradlew bootRun

```

7.4 Проверка кэширования

1. Первый запрос списка спутников:

```
curl http://localhost:8080/api/satellites
```

В логах приложения появится SQL-запрос (из-за промаха кэша).

2. Повторный запрос: тот же curl – SQL-запроса нет, данные взяты из Redis (время ответа значительно меньше).

3. Создание нового спутника:

```
curl -X POST http://localhost:8080/api/satellites \
  -H "Content-Type: application/json" \
  -d '{"name":"Sputnik-X","state":"ACTIVE"}'
```

После этого кэш `satellites::all` полностью сброшен (аннотация `@CacheEvict(allEntries=true)`). Следующий GET-запрос списка снова вызовет SQL и закэширует обновлённый список.

7.5 Проверка graceful degradation

1. Остановите контейнер Redis:

```
docker stop <имя_контейнера_redis>
```

2. Повторите запрос списка спутников. Приложение должно ответить без ошибок (данные берутся напрямую из БД). В логах появятся сообщения `Cache GET error` и т.п., но исключения не выбросятся.

3. Запустите Redis снова: `docker start <имя_контейнера_redis>` – кэш снова заработает.

8 Заключение

Внедрённое кэширование с Redis и Spring Cache Abstraction позволило:

- сократить количество обращений к базе данных для операций чтения;
- гибко управлять временем жизни кэша (разные TTL для разных сущностей);
- автоматически сбрасывать кэш при изменениях данных, сохраняя согласованность;
- обеспечить отказоустойчивость (работа без кэша при недоступности Redis);
- мониторить состояние кэша через стандартные метрики Actuator.

Данный подход рекомендован для высоконагруженных сервисов, где допустима небольшая задержка в актуализации данных (до истечения TTL).